CIBERTEC

Programa IA Generativa Desarrollo de Soluciones con Python

Unidad 2: Conteinerización

OBJETIVOS

- Analizar la importancia de la contenerización.
- Utilizar Docker y docker-compose para crear y gestionar contenedores.
- Diferenciar entre UI y API Rest, utilizando herramientas como Streamlit y FastAPI.

Módulo 4: Docker

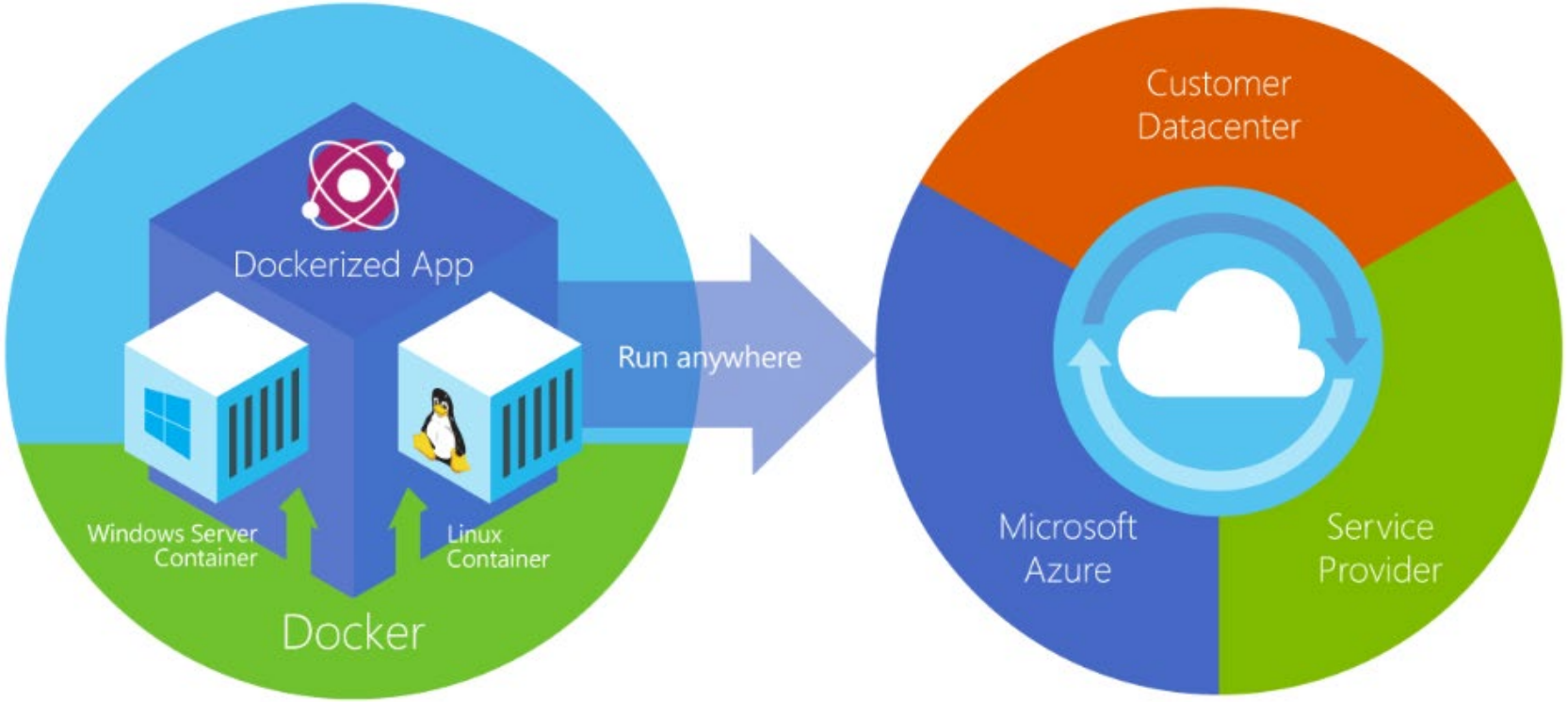
AGENDA

1. Introducción a la contenerización y Docker
2. Dockerización de aplicaciones Python

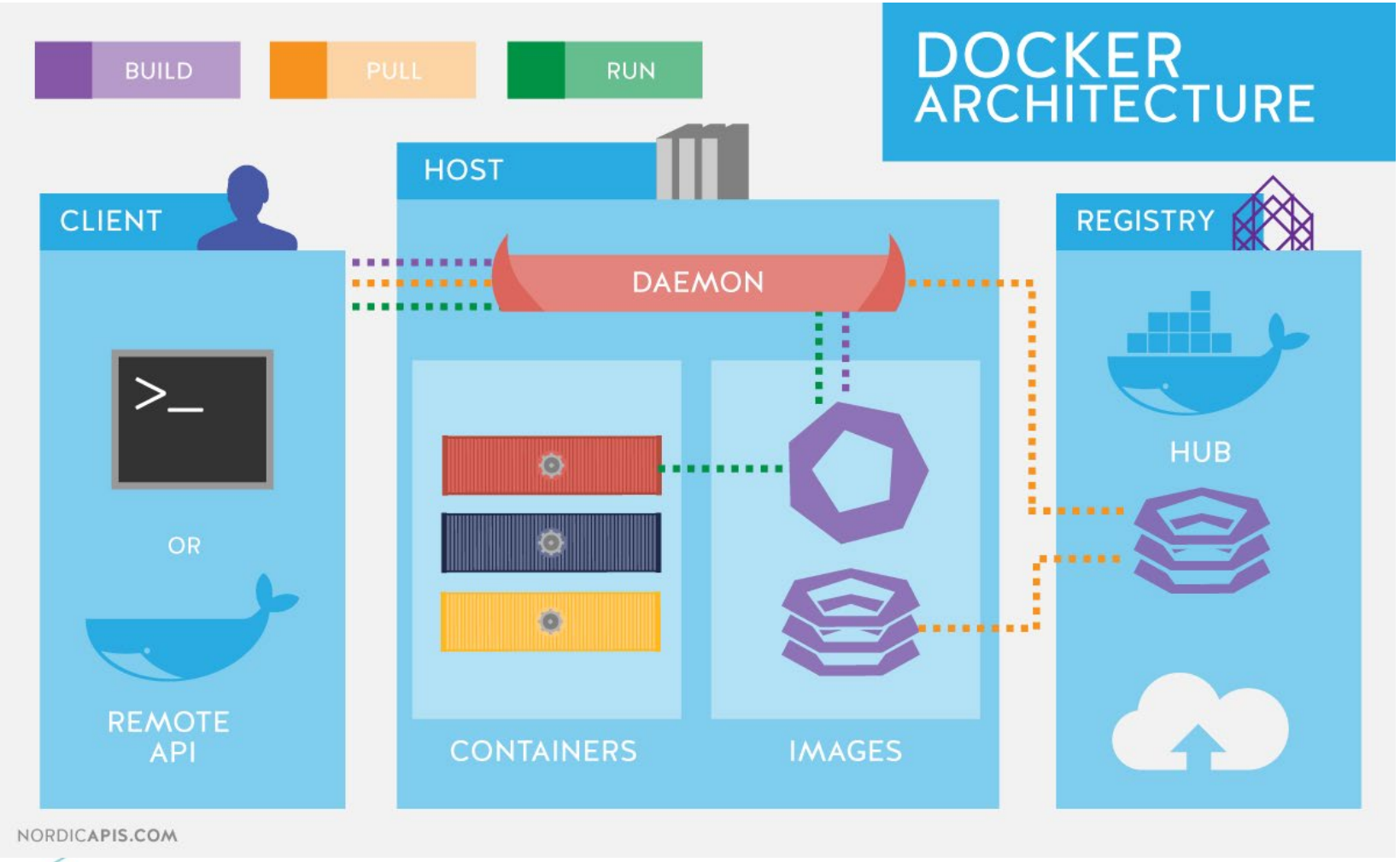
Laboratorio:

Dockerización de app Python **(Evaluación)**

1. INTRODUCCIÓN A LA CONTENERIZACIÓN Y DOCKER



1. INTRODUCCIÓN A LA CONTENERIZACIÓN Y DOCKER



2. DOCKERIZACIÓN DE APLICACIONES PYTHON

Crear un Dockerfile:

- El Dockerfile es un archivo de configuración que describe cómo construir una imagen de contenedor.
- En este documento se especifican las dependencias, comandos de instalación y configuración necesarios para la aplicación Python.

2. DOCKERIZACIÓN DE APLICACIONES PYTHON

```
# Usamos una imagen base de Python
FROM python:3.9

# Directorio de trabajo dentro del contenedor
WORKDIR /app

# Copiamos los archivos de la aplicación al contenedor
COPY . .

# Instalamos las dependencias
RUN pip install -r requirements.txt

# Comando para ejecutar la aplicación
CMD ["python", "app.py"]
```

LABORATORIO Nº 3: DOCKERIZACIÓN DE APP PYTHON

En este laboratorio, podrás crear y gestionar imágenes de Docker y contenedores, aplicando conceptos de contenerización en el desarrollo de aplicaciones.

Tarea 1: Modificar el Dockerfile para incorporar el comando de instalación de dependencias con PIP y el de ejecución de la app Python.

Tarea 2: Probar la construcción de imagen y ejecución de containers mediante la ventana de comandos (terminal). Para ello es necesario que completes el archivo **docker-commands.txt,** con los comandos solicitados.

Módulo 5: UI y API REST

AGENDA

Módulo 5: UI y REST API

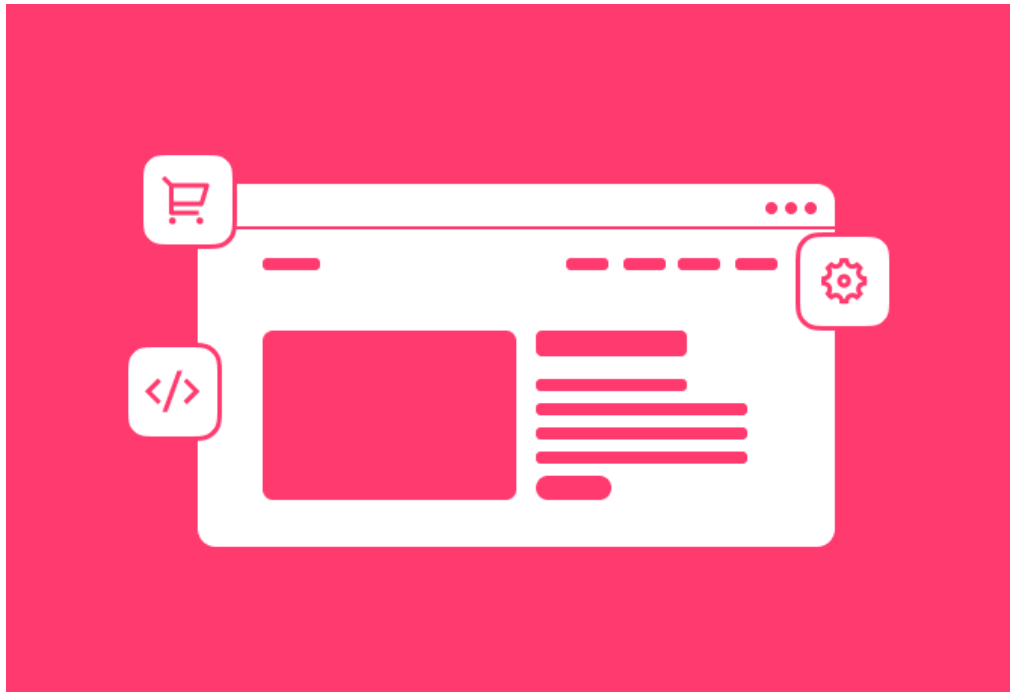
1. Diferenciando entre UI y API REST
2. Herramientas para Desarrollo de UI y API REST
3. Streamlit y FastAPI usando Docker-compose

Laboratorio:

Desplegar Streamlit y FastAPI usando Docker-compose **(Evaluación)**

1. DIFERENCIANDO ENTRE UI Y API REST

La Interfaz de Usuario (UI) se refiere a la parte visual y funcional de una aplicación con la que los usuarios interactúan directamente.

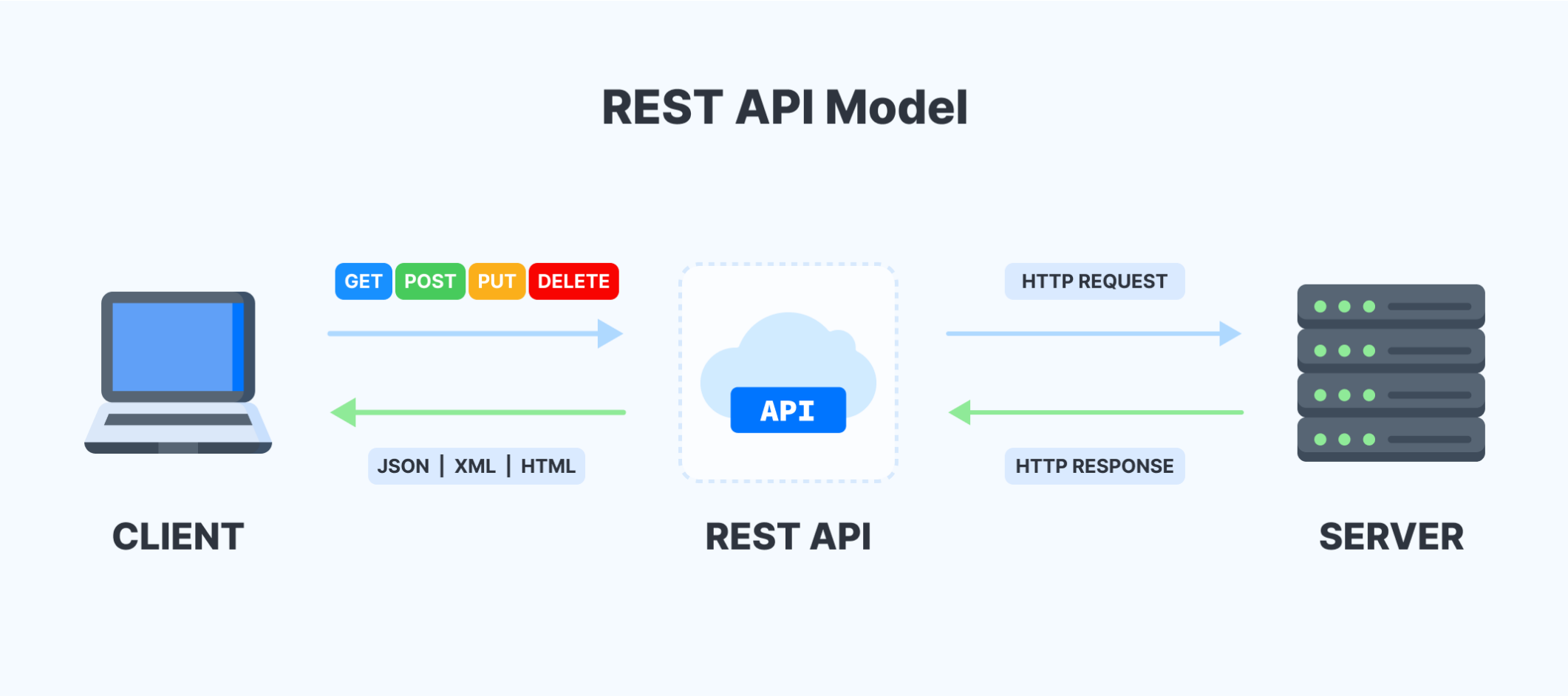


1. DIFERENCIANDO ENTRE UI Y API REST

Características:

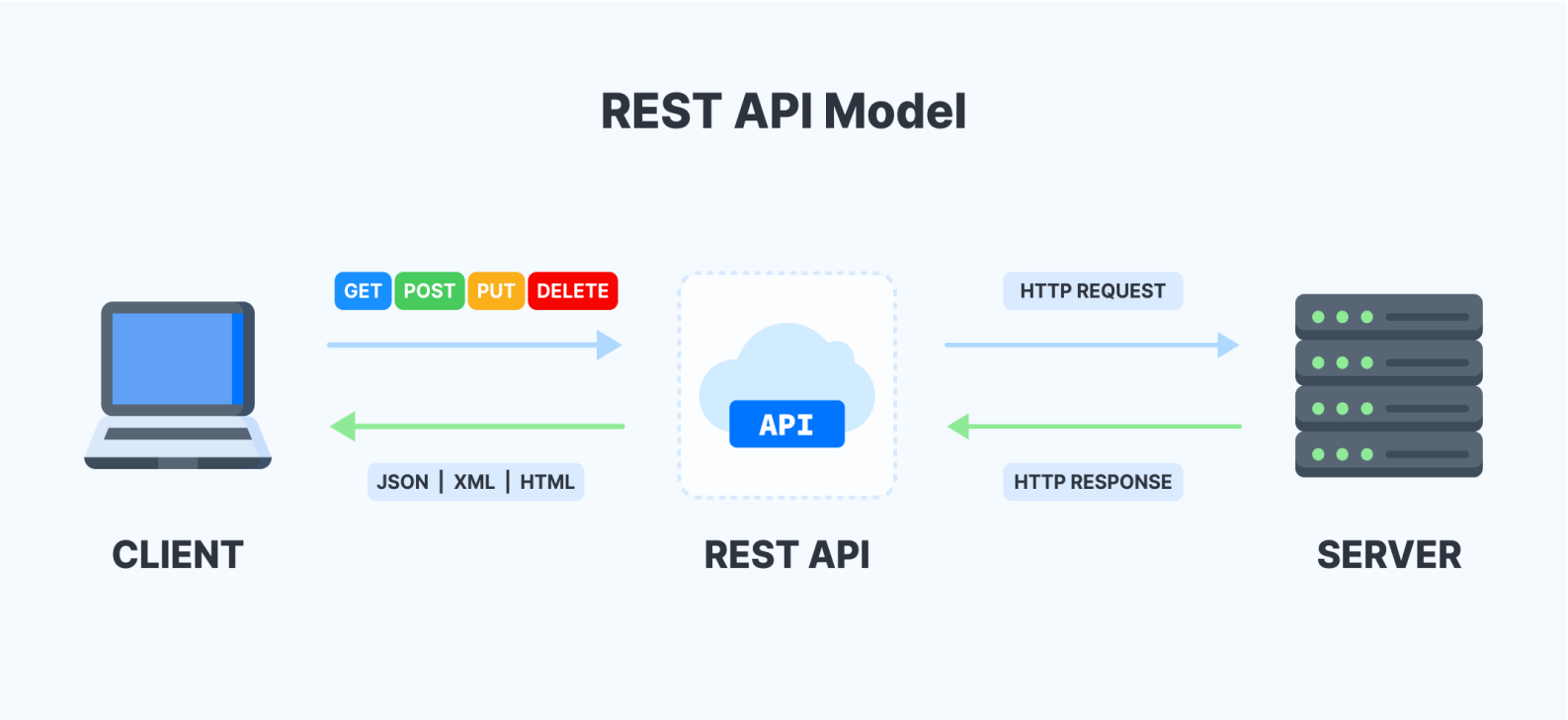
- La UI abarca la arquitectura de información, patrones y elementos visuales que permiten una interacción eficaz con sistemas operativos y software en diversos dispositivos.
- Una UI flexible puede adaptarse a las necesidades del usuario, lo que favorece su éxito a largo plazo.

1. DIFERENCIANDO ENTRE UI Y API REST



1. DIFERENCIANDO ENTRE UI Y API REST

La API REST (Transferencia de Estado Representacional) es una arquitectura popular para desarrollar APIs.



1. DIFERENCIANDO ENTRE UI Y API REST

Características:

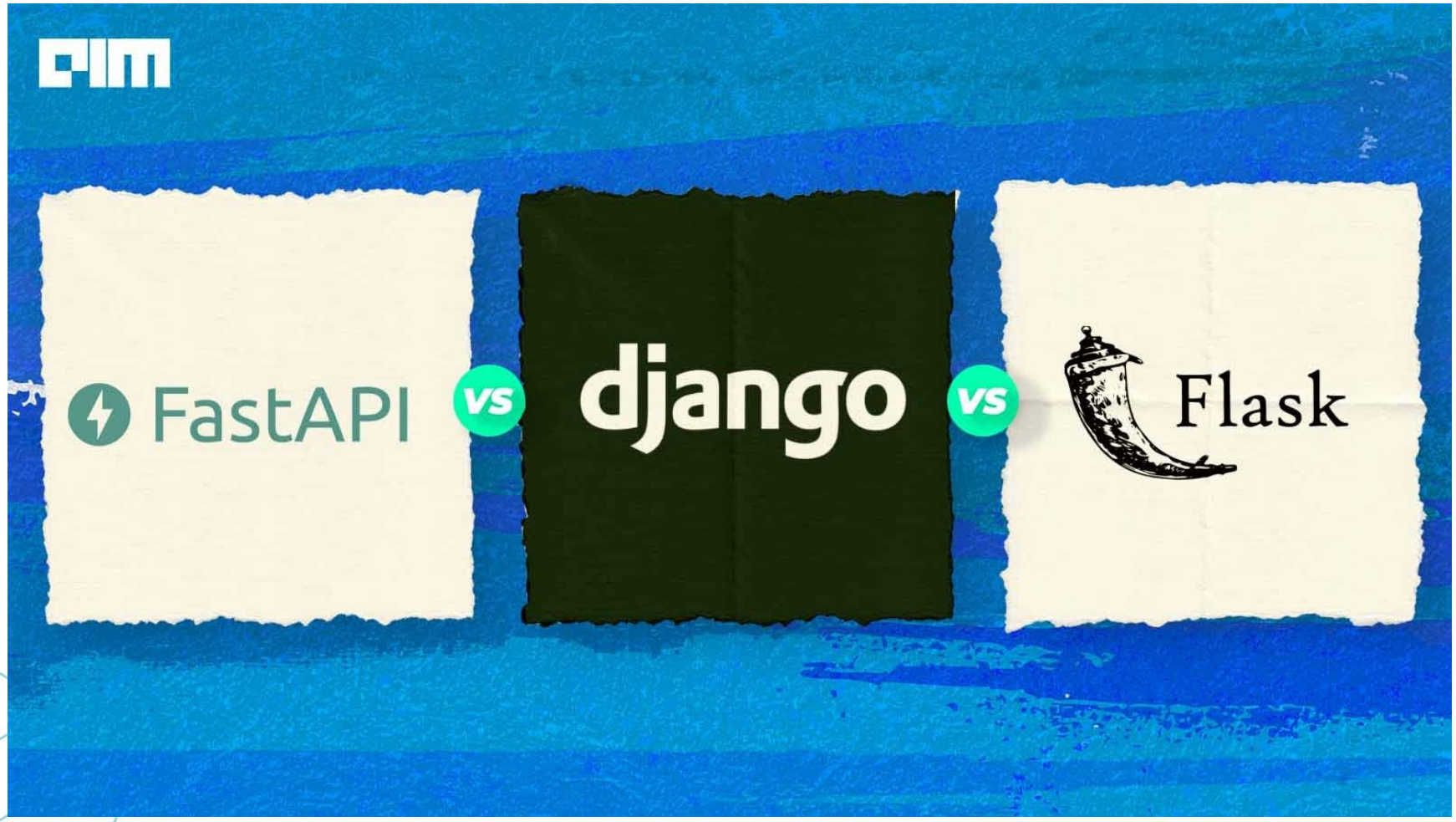
- **Sin Estado:** no almacena datos ni estado entre peticiones.
- **Interfaz Uniforme:** define restricciones arquitectónicas para construir aplicaciones que se comunican a través de HTTP.
- **Métodos HTTP:** utiliza métodos como GET, POST, PATCH, DELETE o PUT, para recuperar datos y hacer peticiones.

1. DIFERENCIANDO ENTRE UI Y API REST

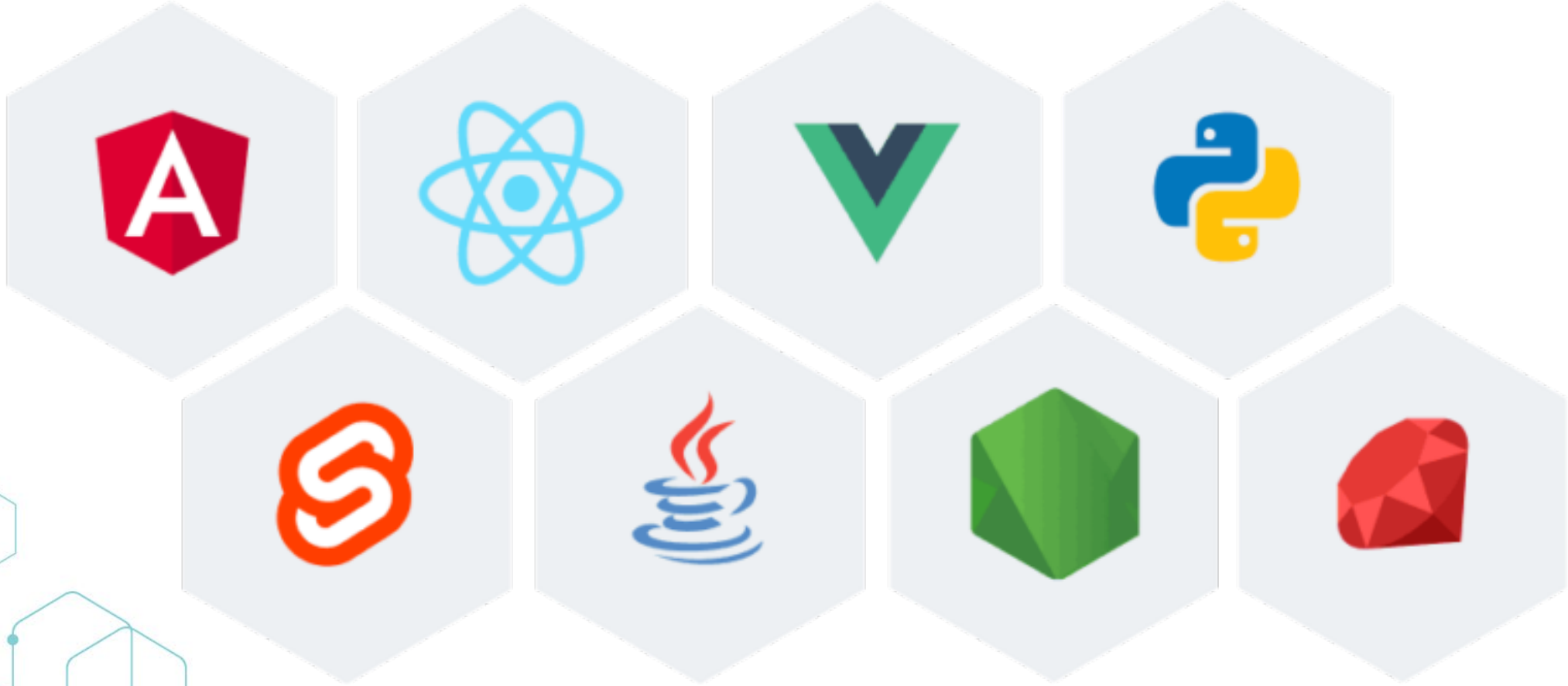
Características:

- **Arquitectura Desacoplada:** la lógica está separada de la presentación.
- **Escalabilidad:** permite ampliar los puntos finales para incorporar lógica más compleja.
- **Almacenabilidad en Caché:** aunque no tiene estado, las respuestas del servidor pueden almacenarse en caché para evitar peticiones redundantes

2. HERRAMIENTAS PARA DESARROLLO DE UI Y API REST

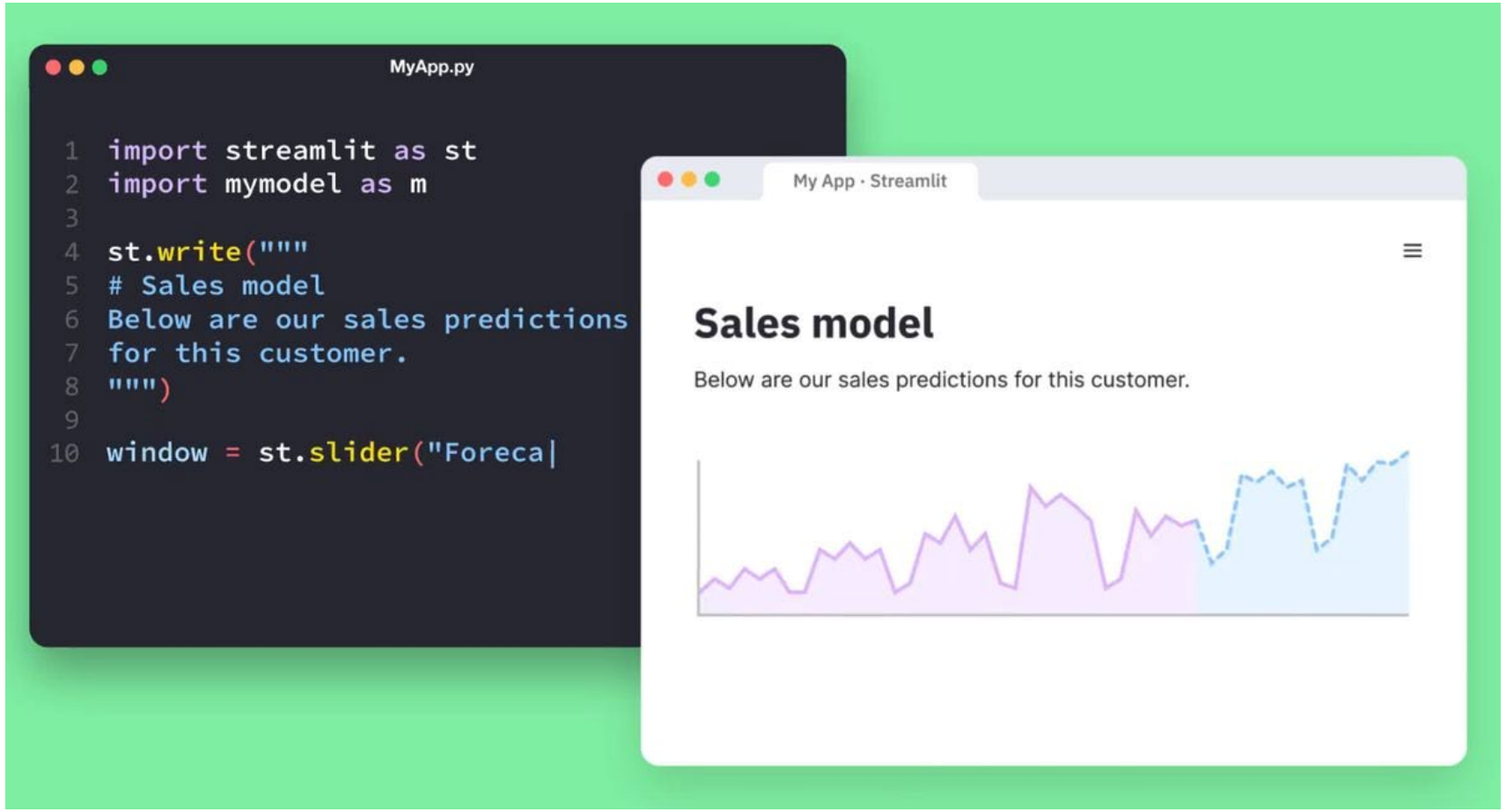


2. HERRAMIENTAS PARA DESARROLLO DE UI Y API REST



2. HERRAMIENTAS PARA DESARROLLO DE UI Y API REST

Streamlit



3. STREAMLIT Y FASTAPI USANDO DOCKER-COMPOSE

LABORATORIO Nº 4: DESPLEGAR STREAMLIT Y FASTAPI USANDO DOCKER-COMPOSE

En este laboratorio, podrás aplicar los conceptos de Docker para levantar aplicaciones en Python, tanto de UI como de RestAPI, y probar dichas aplicaciones, a través de la documentación automática generada por FastAPI mediante Swagger.

Tarea 1: Completar los comandos del Dockerfile llamado **Dockerfile-API,** para ejecutar el script de FastAPI.

Tarea 2: Editar el **Dockerfile-UI** y levantar la aplicación de Streamlit.

TAREA Nº 2:
DEFINICIÓN DE REQUERIMIENTOS. CONTENERIZACIÓN DE PROYECTO BASE

Al finalizar la tarea, lograrás:

- Definir requisitos del proyecto.
- Contenedorizar el proyecto base.
- Documentar requisitos funcionales y no funcionales.
- Configurar el entorno con Docker.
- Crear y gestionar contenedores.
- Preparar ambiente de desarrollo para implementación del proyecto.
- Mejorar portabilidad y gestión de recursos mediante contenedores.

LECTURAS ADICIONALES

Se sugiere revisar los siguientes enlaces para profundizar los conceptos tratados.

RESUMEN

En este módulo, aprendiste sobre la contenedorización con Docker y docker-compose, diferenciando entre UI y API REST, utilizando herramientas como Streamlit y FastAPI.

Además, adquiriste habilidades para desplegar estas aplicaciones con Docker-compose y considerar mejores prácticas para su uso efectivo en entornos de desarrollo y producción.

CIBERTEC